

Extending the Actor Model with Synchronous Delivery: **fast_send** from CPU Threads to FPGA Pipelines

Vincent Maciejewski
M2 Tech, Montreal
mayeski@gmail.com

October 2026

Abstract

In the actor model, actors communicate only by asynchronous send; practical systems add synchronous calls outside the model. We propose adding a synchronous send to the model itself, as a second primitive: **fast_send**, from the open-source actor framework Kaspar-HFT, where the caller waits while the receiver handles the message and gets the reply back as the return value. We give **fast_send** a formal semantics, extending the active-object calculus of De Koster and De Meuter with explicit execution resources, and prove what adding it keeps and what it costs. Isolation is kept: actors still share no state, and each still handles one message at a time. Determinism becomes relative to replies: a handler's outcome depends only on its message and the replies it receives. Liveness becomes conditional: with terminating handlers and fair scheduling, the system can get stuck only when actors wait on each other in a circle (A waits for B while B waits for A), and without **fast_send** it never gets stuck, as in the original model. We prove that **fast_send** is not shorthand for a send followed by a blocking wait: when actors share a thread, **fast_send** completes where that pattern deadlocks. For debugging and deterministic replay, we prove that running every actor, in any language or on any substrate, from one global queue shows only behaviour the real concurrent system can produce, though not all of it, provided no actor called remotely shares its thread. Finally we show that the extended model maps naturally onto hardware. On an FPGA, **send** is a FIFO between hardware processes and **fast_send** is either logic fused into the caller's process or a blocking request to another process. Without **fast_send**, actors fused into one process could not call each other: a send followed by a wait would deadlock. We also cover CPU threads and Rust, and propose a GPU mapping.

1 Introduction

An actor owns private state and communicates only by messages. In the classic formulations [7, 1, 2] there is exactly one way to communicate: an asynchronous send, which places a message in the receiver's mailbox and returns at once. The receiver processes its messages one at a time, each in a *turn*. Because a turn never waits on anything, turns cannot deadlock, and because no state is shared, turns cannot race. De Koster and De Meuter name this the *isolated turn principle*, give an operational semantics for four families of actor languages, and prove the principle for each [6].

Real actor systems nonetheless need request/reply or synchronisation, and they obtain it in one of three ways. The sender sends a request and then blocks until a reply message arrives — the receiver still runs on its own thread (ABCL/1's *now*-type messages [14], ABS calls between cogs [9], Erlang's `gen_server:call`, Akka's `ask` when its future is awaited); or a synchronous call is allowed

only between objects that already share one scheduling unit (ABS calls inside a cog, E’s immediate calls inside a vat [11]); or sender and receiver meet at a rendezvous (Extended Rebeca [13]).

Kaspar-HFT and `fast_send`. Kaspar-HFT¹ [10] is an open-source C++20 actor framework, written for high-frequency trading and used to build a trading system for CME futures: market data decoding, order-book reconstruction, execution algorithms and order routing are all actors. In Kaspar-HFT an actor is a C++ object that registers one handler per message type and holds references to other actors. It offers two ways to deliver a message. `send` is the classic asynchronous send: the message is placed in the receiver’s mailbox, the sender continues, and a thread that serves the receiver later runs its handler. `fast_send` is synchronous: the *sending* thread acquires the receiver’s lock, runs the receiver’s handler inline, bypassing its mailbox, and receives the handler’s reply as the call’s return value. The handler is the same code in both cases and does not depend on which delivery was used — the property the Kaspar-HFT paper calls *receiver transparency*. Kaspar-HFT adds two further extensions that matter here: actor *groups*, which run a set of actors on one thread behind one shared mailbox, and a choice of mailbox implementation per actor. That paper describes the framework and measures its cost; it does not give `fast_send` a formal semantics.

This paper studies `fast_send`, a different form of synchronous communication: a synchronous send in which the caller waits while the receiver handles the message, and gets the reply back as the return value. On a CPU the receiver’s handler simply runs on the caller’s thread. It is neither a send followed by a blocking wait, nor a call confined to one scheduling unit, nor a rendezvous. Synchronous communication in actor systems is not new; what this paper contributes is a precise semantics for this particular primitive, in which execution resources are explicit, and an exact account of what it does to isolation, determinism, liveness, message order and single-queue simulation when those resources are finite (Sections 4 to 6). We also show that it has a distinct mechanism on each runtime considered, just as asynchronous send does (Section 7). On an FPGA this matters most. Hardware offers two ways to connect computations: a FIFO between two independent processes, and logic fused into one process, which is how FPGA designs get their speed. `send` is the first. Only `fast_send` is the second: two actors fused into one process can call each other with `fast_send`, while a send followed by a wait between them deadlocks. With `fast_send` in the model, actors map onto an FPGA naturally; without it, half of what the hardware offers has no counterpart in the model.

In short, `fast_send` is the synchronous form that runs the receiver’s ordinary handler, reaches any actor across scheduling units, returns a value, and completes on a shared execution resource where send-then-wait deadlocks.

Contributions.

1. A thread-aware extension of the active-object calculus of [6] with a `fast_send` rule, in two variants: the callee’s turn runs on the caller’s resource (FAST-LOCAL) or on the callee’s own resource while the caller waits (FAST-REMOTE) (Section 4). Both variants start the same method body as asynchronous delivery, which is receiver transparency stated in the calculus (Remark 3).
2. Its properties, stated against the isolated turn principle (Section 5): isolation is kept (Theorem 6); determinism holds relative to replies (Theorem 7); under terminating handlers and fair scheduling, a turn can be stuck only if its thread can reach a wait-for cycle (Theorem 10), and the original liveness result is the special case without `fast_send` (Corollary 11); a per-thread call-chain check detects exactly the cycles confined to one thread (Proposition 12); a `fast_send` is never

¹Kaspar-HFT is open source: <https://github.com/vincent212/kaspar-hft>.

queued behind the callee’s mailbox (Proposition 13); and `fast_send` is not send-then-wait when resources are shared (Proposition 14).

3. Trace soundness of a single global queue as a simulator for debugging and deterministic replay, across languages and substrates (Theorem 16), its limits (Proposition 18), and an offline check for the deadlocks it cannot exhibit (Proposition 19) (Section 6).
4. A mapping of both primitives onto CPU threads, a Rust runtime and an FPGA actor runtime — on the FPGA, `send` is a FIFO and `fast_send` is fused logic or a blocking request, and fused actors can call each other only with `fast_send` — a proposed mapping onto GPUs, and the mappings between them (Section 7), with their implementation status in Kaspar-HFT (Section 8).

2 Background: turns and the isolated turn principle

De Koster and De Meuter classify actor languages into four families — classic actors, active objects, processes and communicating event loops — and give each a small-step operational semantics [6]: a set of rules, each describing one step a running system can take. A step is called a *reduction*, and a system *reduces* by taking steps.

Intuitively, the semantics is an interpreter written as a list of rules. A snapshot of the whole system at one moment — every actor’s mailbox, data and current code — is a *configuration*. Each *reduction rule* says: if the system is in a configuration of this shape, it may move to that one. One rule, for example, says that an actor that has finished its current message and has another waiting may take it out of its mailbox and start running the matching handler; another says that an actor evaluating a send puts the message at the end of the receiver’s mailbox. Running a program means applying rules, one step at a time. When several rules apply at once — two actors are both ready to run, say — any one of them may be chosen, and the different choices are the different orders in which a concurrent system can interleave. A property proved of the semantics therefore holds for every such order. We build on their active-object calculus, because it matches the programming model of the runtimes studied here: an actor has a fixed interface, a FIFO mailbox and imperative private state.

In that calculus the state of the whole system, a *configuration*, is the set of its actors. Each actor is written $\mathcal{A}\langle\iota_a, \mu, e, O, o\rangle$, a tuple of five parts, and each message $\mathcal{M}\langle msg, \bar{v}\rangle$; Table 1 says what each symbol stands for. In programming terms, an actor is an object (its active object o) whose methods are its message handlers, together with the ordinary data it owns (its heap O), a queue of messages waiting for it (its mailbox μ), and the code it is running right now (e).

Each reduction rule describes one kind of step (Table 2). Actor-local rules reduce one actor and touch only its own state; actor-global rules involve a second actor. Only SEND and SPAWN cross between actors, and neither makes an actor wait.

SEND appends a message to the receiver’s mailbox after deep-copying the passive objects it references (the function `pass`), so no state is shared. RECEIVE starts a turn: when an actor’s expression is a value, the head of its mailbox is removed and the matching method body becomes its expression. A *turn* is the sequence of reductions of one actor between two applications of RECEIVE.

The isolated turn principle then consists of two theorems [6]. *Safety*: the value a turn reduces to is determined up to α -equivalence — that is, up to the choice of names for things created during the run, such as the identifier given to a new object — whatever the interleaving with other actors, and a turn is not interleaved with another turn of the same actor. *Liveness*: a turn never blocks —

Table 1. The symbols of the active-object calculus of [6].

Symbol	Meaning
$\mathcal{A}\langle\iota_a, \mu, e, O, o\rangle$	an actor
ι_a	the actor’s identifier: its address
μ	its mailbox: the messages waiting for it, oldest first
e	the expression it is evaluating: the code it is running now
O	its heap: the ordinary (“passive”) objects it owns, which no other actor can reach
o	its active object: the object whose methods are the actor’s message handlers
$\mathcal{M}\langle msg, \bar{v}\rangle$	a message: the name of the method to run, msg , and its arguments, $\bar{v}$
$\bar{v}$	a list of values; likewise $\bar{e}$, a list of expressions
self	inside a method, the actor’s own identifier
pass	the copy made when a value crosses from one actor to another, so no state is shared

Table 2. The reduction rules of the active-object calculus of [6], which this paper reuses unchanged.

Kind	Rule	Step
actor-local	LET	bind a value to a local variable
	NEW	create an object in the actor’s own heap
	INVOKE	call a method on one of the actor’s own objects
	FIELD-ACCESS	read a field of one of its objects
	FIELD-UPDATE	write a field of one of its objects
	SELF-SEND	put a message in its own mailbox
	RECEIVE	take the next message from its mailbox and start handling it: a turn begins
actor-global	SEND	put a message in another actor’s mailbox, with the data it references copied, so nothing is shared
	SPAWN	create a new actor

every reducible expression of a turn can take a step — so turns are free of deadlock. The calculi contain no blocking operation; this is what makes liveness hold.

Two features of this setting matter below. The calculus has no notion of the thread or processor that performs a reduction: each actor reduces independently, as if it had its own. And every communication is a SEND. A synchronous call needs the first to be made explicit and adds a second kind of communication.

3 Synchronous communication in actor systems

Three reasons are usually given for leaving synchronous calls out of the actor model. *Order*: in Hewitt’s model messages are guaranteed to arrive but in no particular order, so “handle this now, before what I do next” has no meaning. *Deadlock*: an actor that handles one message at a time and waits inside a turn can wait on itself, directly or through others. *Distribution*: across machines, a synchronous call can wait forever on a failed peer.

Where systems provide synchronous communication anyway, they do so in one of three ways.

Wait on the receiver’s thread. ABCL/1 distinguishes *past* (asynchronous), *now* (send and wait for the reply) and *future* messages [14]. In ABS [9], a synchronous call $v = o.m(e)$ between objects in different cogs abbreviates $x = o!m(e)$; $v = x.get$: an asynchronous call followed by a blocking `get`, which “blocks all execution in the object until the return value is available”. Erlang’s `gen_server:call`, and Akka’s `ask` when the caller awaits the future it returns, are library forms of the same pattern, usually with a timeout. In all of these the receiver’s turn runs on the receiver’s own thread; the caller’s thread waits.

Call within one scheduling unit. In ABS, a synchronous call between objects in the same cog has “the reentrant semantics known from, e.g., Java threads” [9]: control passes to the callee on the cog’s single processor. In E, immediate calls are allowed only on near references, to objects in the same vat; between vats only eventual sends exist [11]. These calls run on the caller’s processor, but they never cross a scheduling unit, so they need no mutual exclusion; and ABS allows re-entry into an object that is already executing.

Rendezvous inside a component. Extended Rebeca [13] groups actors (“rebecs”) into components. Components communicate only asynchronously; inside a component, rebecs may also exchange synchronous messages. A synchronous message has a signature but no handler: it is a “handshake between the execution of a send-statement by the caller and a receive statement by the callee”, and both sides block until they meet at that point; the data flows from sender to receiver only, and a rebec may not send a synchronous message to itself. The receiver takes part explicitly, by executing a `receive` statement inside one of its methods.

Expressiveness. Honda and Tokoro [8] and Boudol [4] showed that synchronous communication can be encoded in the asynchronous π -calculus. `fast_send` therefore adds nothing to what can be computed. The encodings assume, however, that every process makes progress independently — that execution resources are unbounded. Proposition 14 shows that once resources are shared, as in an actor group or a cog, send-then-wait deadlocks where `fast_send` completes. The difference between the primitives is a difference in who executes a turn and when.

Where `fast_send` sits. `fast_send` runs the receiver’s turn on a resource the caller is willing to wait on — on a CPU, the caller’s own thread — across scheduling units, under the receiver’s mutual exclusion, and without re-entry: an actor already in a turn cannot be entered again, and a call that tries waits, possibly forever. It combines the “run it here” of calls within a scheduling unit with the “any actor” of send-then-wait, and pays for it with a deadlock case that calls within a scheduling unit do not have. Unlike a rendezvous, the receiver does not take part: its ordinary handler runs as a whole turn and its result comes back as a value. Table 3 summarises the differences; the rest of the paper makes them precise.

4 A thread-aware calculus with `fast_send`

To reason about `fast_send` we need a precise description of what it does, step by step, in the same terms as the rest of the actor model. This section gives one. It does so by extending the calculus of Section 2 rather than starting afresh, so that everything the original calculus says about `send` still holds, and the new rules are the only place `fast_send` can change anything.

Table 3. Synchronous communication in actor and active-object systems. “Busy callee”: what happens when the callee is in a turn when the call is made.

Mechanism	Who runs the callee’s code	Reach	Reply value	Re-entry	Busy callee
Send-then-wait (ABCL/1 <i>now</i> , ABS between cogs, <code>gen_server:call</code>)	the callee’s own processor, in a later turn	any actor	yes	no	request queued behind its mailbox
Call in one unit (ABS inside a cog, E immediate call)	the caller’s processor	same cog / vat	yes	yes	not applicable: one processor
Rendezvous (Extended Rebeca)	both, at a receive statement in the callee	same component	no	no self-send	sender waits for the receive
fast_send (this paper)	the caller’s resource, or one dedicated to the callee	any actor	yes	no	caller waits for the turn to end; mailbox bypassed

The original calculus has no notion of *who* runs an actor’s code: each actor simply takes steps, as if it had a processor of its own. That is enough for asynchronous send, which never waits, but not for **fast_send**. Whether a synchronous call completes depends on which thread runs the callee’s handler and on what else that thread is doing: on a thread that is free it completes, and on a thread that is itself waiting it may never start. So the first change is to make execution resources — threads — part of the model, and to keep, for each thread, the stack of handlers it is in the middle of, as a call stack does in an ordinary program. The second is to add **fast_send** itself, in two forms: the callee’s handler runs on the caller’s thread, or on a thread of its own while the caller waits.

Concretely, we extend the active-object calculus of [6] in three ways: execution resources become explicit, an actor’s current expression moves from the actor into a *frame* on a resource’s stack, and a **fast_send** expression is added. We call the resources *threads*; on an FPGA they are hardware processes (Section 7).

4.1 Semantic entities and syntax

$$\begin{aligned}
K \in \text{Configuration} &::= \langle A, T \rangle \\
a \in A \subseteq \text{Actor} &::= \mathcal{A} \langle \iota_a, \mu, O, o, \theta \rangle \\
t \in T \subseteq \text{Thread} &::= \mathcal{T} \langle \theta, S \rangle \quad S ::= \epsilon \mid S \cdot f \\
f \in \text{Frame} &::= \mathcal{F} \langle \iota_a, e, \kappa \rangle \quad \kappa ::= \text{turn} \mid \text{ret}(\rho)
\end{aligned}$$

An actor has an identifier, a FIFO mailbox μ , a heap O , an active object o , and a *home thread* θ . Several actors may share a home thread; that is an actor group. A thread has an identifier and a stack S of frames, top on the right. A frame records which actor it belongs to, the expression being evaluated, and why it exists: **turn** for a turn started from the mailbox, or **ret**(ρ) for a turn started by the **fast_send** call ρ , whose value must be returned. Call identifiers ρ are fresh. Table 4 summarises the new symbols. In programming terms, a thread is a call stack; each frame on it is one actor’s handler in progress, and a **fast_send** that runs on the caller’s thread pushes the callee’s handler on top of the caller’s, exactly like a function call.

Table 4. The symbols this paper adds to the calculus.

Symbol	Meaning
$K = \langle A, T \rangle$	a configuration: the set of actors A and the set of threads T
θ	a thread's identifier; also an actor's <i>home thread</i> , the one that serves its mailbox
$\mathcal{T}\langle \theta, S \rangle$	a thread, with its stack S of frames, top on the right (ϵ is the empty stack)
$\mathcal{F}\langle \iota_a, e, \kappa \rangle$	a frame: actor ι_a 's handler in progress, running expression e
$\kappa = \text{turn}$	the frame is a turn started from the actor's mailbox
$\kappa = \text{ret}(\rho)$	the frame is a turn started by fast_send call ρ ; its result goes back to the caller
ρ	a fast_send call's identifier, new for each call
$\text{fastsend}(\iota_b, \text{msg}, \bar{v})$	the fast_send expression: call actor ι_b 's handler msg with arguments $\bar{v}$
$\text{wait}(\rho)$	where a caller's expression waits for the result of call ρ
$\text{busy}_K(\iota_a)$	actor ι_a has a handler in progress somewhere in K
$\text{body}(a, \text{msg}, \bar{v})$	the code of actor a 's handler for msg , with its arguments filled in
v	a value: an expression that has finished computing
$m \cdot \mu, S \cdot f$	the mailbox μ with message m at its head; the stack S with frame f pushed on top
$K \longrightarrow K'$	configuration K can take one step, to K'
$K \longrightarrow^* K'$	K can reach K' in zero or more steps

The expression syntax of the active-object calculus is extended with one form, and the run-time syntax with one more:

$$e ::= \dots \mid \text{fastsend}(e, \text{msg}, \bar{e}) \qquad e ::= \dots \mid \text{wait}(\rho) \quad (\text{run time})$$

Read “ $e ::= \dots \mid X$ ” as “an expression is anything it was before, or X ”: a program may now contain $\text{fastsend}(e, \text{msg}, \bar{e})$, and while a call is in progress the runtime puts $\text{wait}(\rho)$ where the call was. A programmer never writes $\text{wait}(\rho)$.

An *evaluation context* $\mathcal{E}$ is an expression with one hole in it, marking the part to be computed next; $\mathcal{E}[e']$ is the expression with e' in the hole. In $x + \text{fastsend}(\iota_b, \text{msg}, 1)$, for instance, the context is $x + []$ and the **fast_send** is what runs next. Evaluation contexts fix the order in which an expression is computed; they extend the originals left to right through the arguments of **fastsend**, so the arguments are computed before the call is made. No rule reduces $\text{wait}(\rho)$: a frame whose expression has the form $\mathcal{E}[\text{wait}(\rho)]$ is stopped at the call and is called *waiting*.

Definition 1 (Busy). Actor ι_a is *busy* in K , written $\text{busy}_K(\iota_a)$, if some thread of K has a frame of ι_a on its stack.

An actor's active object is written $o = \mathcal{O}\langle \iota_o, \text{cls}, \bar{v}_f \rangle$: its identifier ι_o , its class cls , and the values of its fields $\bar{v}_f$. For such an actor and a message $\mathcal{M}\langle \text{msg}, \bar{v} \rangle$, let $\text{body}(a, \text{msg}, \bar{v})$ be the body of method msg of cls with the fields, the parameters and **self** replaced by their values — the code the handler will run, ready to go — exactly as in **RECEIVE** and **INVOKE** of [6].

$$\begin{array}{c}
\frac{t = \mathcal{T}\langle\theta, \epsilon\rangle \quad a = \mathcal{A}\langle\iota_a, \mathcal{M}\langle msg, \bar{v}\rangle \cdot \mu, O, o, \theta\rangle \quad \neg \text{busy}_K(\iota_a)}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, \mathcal{F}\langle\iota_a, \text{body}(a, msg, \bar{v}), \text{turn}\rangle\rangle][a \mapsto \mathcal{A}\langle\iota_a, \mu, O, o, \theta\rangle]} \text{RECEIVE} \\
\\
\frac{t = \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, v, \text{turn}\rangle\rangle}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, S\rangle]} \text{TURN-END} \qquad \frac{\text{Call}(t, \iota_b, \rho)}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, S \cdot W_\rho \cdot C_\rho\rangle]} \text{FAST-LOCAL} \\
\\
\frac{\text{Call}(t, \iota_b, \rho) \quad \theta_b \neq \theta \quad t_b = \mathcal{T}\langle\theta_b, \epsilon\rangle}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, S \cdot W_\rho\rangle][t_b \mapsto \mathcal{T}\langle\theta_b, C_\rho\rangle]} \text{FAST-REMOTE} \\
\\
\frac{t' = \mathcal{T}\langle\theta', S' \cdot \mathcal{F}\langle\iota_b, v, \text{ret}(\rho)\rangle\rangle \quad t = \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[\text{wait}(\rho)], \kappa\rangle \cdot S''\rangle \quad v' = \text{pass}(v)}{K \longrightarrow K[t' \mapsto \mathcal{T}\langle\theta', S'\rangle][t \mapsto \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[v'], \kappa\rangle \cdot S''\rangle]} \text{FAST-RETURN}
\end{array}$$

Figure 1. Thread rules. $K[t \mapsto t']$ replaces thread t by t' (and likewise for actors). The two **fast_send** rules share the premise $\text{Call}(t, \iota_b, \rho)$, which holds when $t = \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[\text{fastsend}(\iota_b, msg, \bar{v})], \kappa\rangle\rangle$, $\neg \text{busy}_K(\iota_b)$ and ρ is fresh; with $b = \mathcal{A}\langle\iota_b, \mu_b, O_b, o_b, \theta_b\rangle$ and $\bar{v}' = \text{pass}(\bar{v})$, the caller's waiting frame is $W_\rho = \mathcal{F}\langle\iota_a, \mathcal{E}[\text{wait}(\rho)], \kappa\rangle$ and the callee's frame is $C_\rho = \mathcal{F}\langle\iota_b, \text{body}(b, msg, \bar{v}'), \text{ret}(\rho)\rangle$. In **FAST-RETURN**, t and t' may be the same thread, in which case S'' is empty and the returned frame was directly above the waiting one.

4.2 Reduction rules

The actor-local rules of [6] apply unchanged to the *top* frame of any thread, reading and writing the heap of the frame's actor. **SEND** and **SELF-SEND** likewise apply to a top frame and append to the receiver's mailbox. **SPAWN** creates an actor with an empty mailbox and a home thread, either an existing one or a fresh $\mathcal{T}\langle\theta', \epsilon\rangle$. **RECEIVE** and the end of a turn become thread rules, and three rules are new (Figure 1).

Each rule in Figure 1 is read as “if what is above the line holds, the system may take the step below the line”. Above the line are the conditions, for example that a thread's stack is empty; below the line is the step, written $K \longrightarrow K'$, where K' is K with some threads or actors replaced, as $K[t \mapsto t']$ denotes.

- **RECEIVE**: an idle thread starts a turn of one of its idle actors whose mailbox is not empty. With several actors on one thread, the choice among them is free.
- **TURN-END**: a turn started from the mailbox that has reduced to a value is removed. Its value is discarded; a reply to a **send** is an explicit **send**.
- **FAST-LOCAL**: the caller's frame becomes waiting, and the callee's turn is pushed on the *caller's* stack. This is **fast_send** on one CPU thread, inside one FPGA process, or from an FPGA actor to a CPU actor, whose handler runs on a host thread acting for the caller.
- **FAST-REMOTE**: the caller's frame becomes waiting, and the callee's turn starts on the callee's own, idle, home thread. This is **fast_send** to an actor on another resource that the caller waits on: a CPU thread calling into an FPGA process, one FPGA process calling another, or a call to another machine.
- **FAST-RETURN**: a callee turn that has reduced to a value is removed and its value, copied, replaces the $\text{wait}(\rho)$ of its caller.

Two different conditions govern a **fast_send**, and the distinction matters throughout. *Actor availability*: both rules require the callee actor to be idle, that is, to have no frame on any thread.

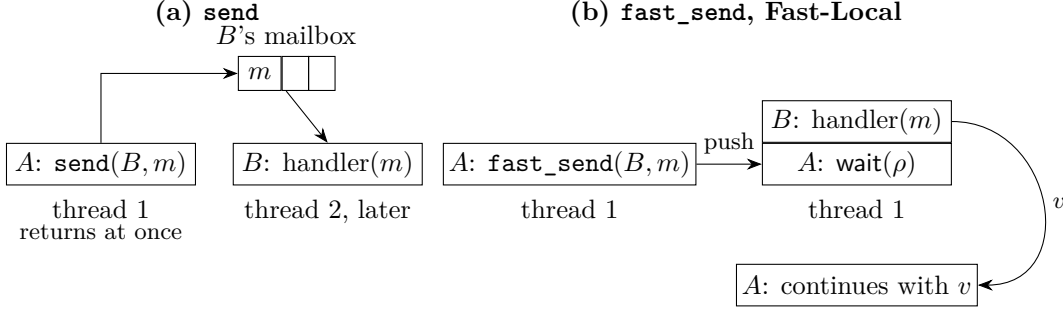


Figure 2. (a) `send` enqueues the message and returns; the receiver’s handler runs later on a thread that serves it. (b) `fast_send` under FAST-LOCAL: the caller’s frame waits, the callee’s turn is pushed on the caller’s own thread, and its value v is returned when it ends. The callee’s mailbox is not used.

Resource availability: FAST-REMOTE additionally requires the callee’s home thread to be idle. An actor can be idle while its home thread is busy running another actor’s turn; FAST-LOCAL then still applies, because it does not use the callee’s thread, but FAST-REMOTE does not. When the condition of the chosen rule fails, the calling frame is *blocked*. The callee’s mailbox is never consulted. A system may use FAST-LOCAL for some pairs of actors and FAST-REMOTE for others; every result below holds for any such mixture. Figure 2 shows `send` and FAST-LOCAL side by side.

Remark 2 (What `fast_send` does not provide). `fast_send` is not a function call. It does not enter an actor that is already in a turn: re-entry is excluded, and a call to a busy actor waits. It gives no guarantee that the call completes (Theorem 10), and hence no deadlock freedom. It is not a priority mechanism: it is not queued behind the callee’s mailbox (Proposition 13) but does not interrupt a turn in progress. It carries no timing guarantee, no failure handling — a call to an actor whose resource has failed waits forever — and no transactional semantics across the actors a turn calls.

Remark 3 (Receiver transparency). A method body cannot tell how its turn started: RECEIVE, FAST-LOCAL and FAST-REMOTE all install the same `body(b, msg,  $\bar{v}$ )`. This is the calculus counterpart of receiver transparency in Kaspar-HFT [10]. It is a property of the interface a turn is given, not a claim about what code in a real language can observe about its execution context.

5 Properties

With the rules fixed, we can ask what adding `fast_send` does to the guarantees that make the actor model attractive in the first place. De Koster and De Meuter’s isolated turn principle (Section 2) has two halves, and we take them in turn.

- *Isolation* (Section 5.1). Does an actor still handle one message at a time, with no other code touching its state? Yes: `fast_send` never lets a second handler of the same actor start, and never gives one actor access to another’s data.
- *Determinism* (Section 5.2). Is a handler’s outcome still fixed by its input? Yes, once the replies it receives count as input.
- *Liveness* (Section 5.3). Does every handler still finish? Not always: a caller can wait for ever. We show exactly when — only when waits form a circle — and that without `fast_send` the original guarantee comes back.

Two further results say how `fast_send` differs from what it might be mistaken for. Section 5.4 states how it interacts with message order, and Section 5.5 shows that it is not a send followed by a blocking wait: the two behave differently when actors share a thread.

The statements are about every configuration the rules can reach. Throughout, $K_0 \longrightarrow^* K$ ranges over configurations reachable from an initial configuration with one actor, as in [6], whose single thread holds no frames.

5.1 Isolation

Isolation is what lets a programmer reason about one actor at a time: while a handler runs, nothing else changes the actor’s state, and the handler cannot reach any other actor’s state. A synchronous call might seem to threaten this, since the caller’s handler is suspended in the middle while another handler runs. This subsection shows that it does not. The argument is a single invariant — every actor has at most one handler in progress anywhere in the system — from which isolation follows directly.

Lemma 4 (One frame per actor). *In every reachable configuration, each actor has at most one frame, and every frame reads and writes only the heap of its own actor.*

Proof. Initially there are no frames. Only RECEIVE, FAST-LOCAL and FAST-REMOTE create frames, and each creates a frame for an actor that is not busy, that is, has no frame. TURN-END and FAST-RETURN only remove frames; the other rules rewrite the expression of an existing top frame. The heap discipline is that of the active-object calculus: field access and update apply to the heap of the frame’s actor, and values crossing actors (SEND, the two `fast_send` rules, FAST-RETURN) are copied by `pass`. $\square$

Definition 5 (Turn). A turn of ι_a is the sequence of reductions from the creation of a frame of ι_a (by RECEIVE, FAST-LOCAL or FAST-REMOTE) to its removal (by TURN-END or FAST-RETURN).

Theorem 6 (Isolation). *No two turns of the same actor overlap, and during a turn of ι_a the heap of ι_a is changed only by reductions of that turn’s frame.*

Proof. Immediate from Lemma 4: a second turn would need a second frame. $\square$

This is the safety half of the isolated turn principle, and it holds with `fast_send`. A turn that makes a `fast_send` is suspended while the callee runs, but no other turn of the same actor can run in the meantime, and the callee cannot touch the caller’s state.

5.2 Determinism relative to replies

Determinism says that a handler’s result is fixed by what it is given, whatever the other actors are doing at the same time; it is what makes a handler testable on its own. With `fast_send` a handler receives something new partway through: the reply to each call it makes, whose value depends on the callee’s state at that moment. This subsection shows that the replies are the only new source of variation. A handler is as deterministic as before, once its replies are counted among its inputs.

In [6] a turn’s value is determined, because a turn receives no input after it starts. With `fast_send` it does: each FAST-RETURN substitutes a value that depends on the callee’s state, which other turns may have changed before the call.

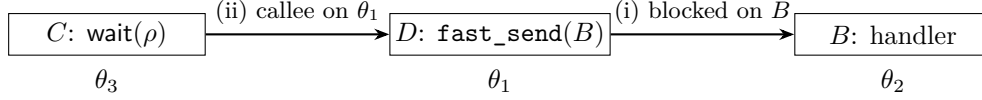


Figure 3. Wait-for edges in one configuration. C on θ_3 made a `fast_send` to D under `FAST-REMOTE`, so D 's turn (a frame marked `ret( $\rho$ )`) runs on θ_1 and θ_3 waits for it: an edge of kind (ii). D 's top frame is a `fast_send` to B , which is busy on θ_2 : an edge of kind (i). θ_2 has no outgoing edge, so its top frame can be reduced; when B 's turn ends, D 's call can start, and the path shortens from the right. With no cycle on the path, every thread on it eventually progresses (Theorem 10).

Theorem 7 (Determinism relative to replies). *Consider two turns of the same actor that start from α -equivalent heaps and expressions. If the values substituted into them by `FAST-RETURN`, in order, are α -equivalent, then the two turns end with α -equivalent values and heaps and issue α -equivalent `send` and `fast_send` requests in the same order.*

Proof. As the safety proof of [6], by induction on the reductions of the turn's frame: evaluation contexts fix the order of reduction, and every actor-local rule produces α -equivalent results from α -equivalent inputs. The new cases are the two `fast_send` rules, which rewrite the frame to $\mathcal{E}[\text{wait}(\rho)]$ and perform no reduction on it until `FAST-RETURN`, whose substituted value is α -equivalent by hypothesis. Reductions of other frames do not touch this turn's heap (Lemma 4). $\square$

A turn that makes no `fast_send` satisfies the original safety theorem unchanged. A turn that does is deterministic as a function of its replies, as it would be if the replies were messages; what `fast_send` removes is the possibility of any other message of the same actor being handled between the request and the reply.

5.3 Liveness

Liveness says that every handler that starts eventually finishes. In the original model this holds because nothing ever waits. A `fast_send` waits, so liveness can fail, and this subsection says exactly when. We first define which thread is waiting for which, as a graph, then show that a handler can be stuck for ever if and only if its thread leads, in that graph, to a circle of threads each waiting for the next. Without `fast_send` the graph has no edges, so the original guarantee is a special case. We end with what a simple check on one thread can and cannot detect.

Definition 8 (Blocked frame, wait-for graph). A top frame $\mathcal{F}(\iota_a, \mathcal{E}[\text{fastsend}(\iota_b, msg, \bar{v})], \kappa)$ on thread θ is *blocked* in K if the `fast_send` rule used for that call does not apply to it. The *wait-for graph* WF_K has the threads of K as nodes and an edge $\theta \rightarrow \theta'$ when either

- (i) the top frame of θ is blocked on ι_b and θ' holds the frame of ι_b ; or it is blocked by `FAST-REMOTE` because ι_b 's home thread θ' is not idle; or
- (ii) θ holds a waiting frame $\mathcal{E}[\text{wait}(\rho)]$ whose callee frame `ret( $\rho$ )` is on $\theta' \neq \theta$.

Figure 3 shows both kinds of edge. Every edge leaves a thread that cannot progress until something on the target thread is removed. A thread whose top frame is waiting has an edge of kind (ii) unless the callee is above it on the same stack, in which case the thread is not stuck: its top frame is the callee's.

The liveness result is not a termination theorem for arbitrary actor programs. We assume that every handler terminates once the `fast_send` calls it makes have returned, and that scheduling is fair. Fairness has two parts. The first is the usual one: a thread whose top frame can be reduced

is eventually reduced. The second concerns a blocked call: when a busy callee's turn ends, the calculus lets its home thread start its next mailbox turn before the waiting `fast_send` applies, and this can repeat for ever, starving the caller. We therefore also assume that a `fast_send` that becomes applicable infinitely often is eventually applied. A runtime whose lock gives no fairness guarantee does not provide this; the preference for pending calls in Proposition 13 does. Under these assumptions the only source of non-termination that remains is cyclic waiting, which `fast_send` introduces; a handler that loops forever violates the assumption and is outside the result.

Assumption 9 (Terminating handlers, fair scheduling). (a) Every method body terminates when its `fast_send` calls return. (b) Every thread whose top frame can be reduced is eventually reduced. (c) A blocked `fast_send` whose rule becomes applicable infinitely often is eventually applied.

Theorem 10 (Liveness and wait-for cycles). *Under Assumption 9, a turn in a reachable configuration K fails to complete in some fair continuation of K if and only if, in some configuration reachable from K , its thread has a path in the wait-for graph to a cycle.*

Proof. ($\Leftarrow$) Take the cycle $\theta_1 \rightarrow \dots \rightarrow \theta_n \rightarrow \theta_1$. Each θ_i waits for a frame on θ_{i+1} to be removed. That frame is below the top of θ_{i+1} or is its top; in either case its removal needs θ_{i+1} to progress, which needs a frame on θ_{i+2} removed, and so on around the cycle. No rule removes a frame of a thread whose top frame is blocked or waiting, so none of the θ_i ever progresses, and a thread with a path to the cycle waits on it forever.

($\Rightarrow$) Under Assumption 9, a frame that can be reduced is eventually reduced and, its body terminating, eventually removed. So a turn that never completes must, from some point on, sit on a thread that is permanently blocked or waiting, and we show that this requires a path to a cycle. Suppose not: in every configuration reachable from K , the path from θ in WF_K is finite and acyclic. Its last thread θ^* has no outgoing edge, so its top frame is neither blocked nor waiting on another thread, and it is reduced until it is removed by TURN-END or FAST-RETURN. Each such removal either brings θ^* closer to removing the frame its predecessor on the path waits for — there are finitely many frames above it — or removes that frame, after which the predecessor's waiting frame resumes or its blocked `fast_send` becomes applicable, and by part (c) of Assumption 9 is eventually applied. Induction on the length of the path then shows that θ itself progresses, contradicting the assumption that its turn never completes. $\square$

Corollary 11 (Liveness without `fast_send`). *If a program never evaluates `fastsend`, every turn completes under Assumption 9.*

Proof. Without `fast_send` there are no blocked or waiting frames, so the wait-for graph has no edges. $\square$

This is the liveness half of the isolated turn principle, recovered as the case without `fast_send`. With `fast_send`, liveness is a property of the program's pattern of synchronous calls rather than of the model.

Proposition 12 (What a per-thread call chain detects). *Suppose a thread checks, before each `fast_send` to ι_b , whether ι_b has a frame on its own stack. The check succeeds exactly for the self-loops $\theta \rightarrow \theta$ of the wait-for graph. It does not detect cycles of length two or more.*

Proof. A self-loop arises when the top frame of θ is blocked on an actor whose frame is on θ itself, which is what the check tests. A cycle through two threads $\theta_1 \rightarrow \theta_2 \rightarrow \theta_1$ arises when the top frame of each is blocked on an actor whose frame is on the other: each check inspects only its own stack, finds the callee absent, and passes. For example, thread 1 runs a `fast_send` to A whose body

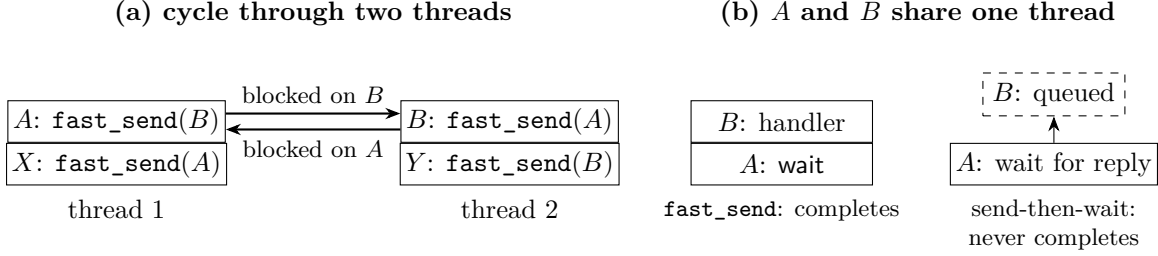


Figure 4. (a) A wait-for cycle through two threads: each top frame is blocked on an actor whose frame is on the other thread. A check of each thread's own stack passes on both (Proposition 12). (b) When A and B share a thread, `fast_send` pushes B 's turn on it and completes; `send-then-wait` leaves the request in B 's mailbox behind A 's waiting turn, and B never runs (Proposition 14).

makes a `fast_send` to B , while thread 2 runs a `fast_send` to B whose body makes a `fast_send` to A . $\square$

Figure 4(a) shows the two-thread case. A per-thread detector, such as the call-chain test designed in [10],² therefore turns same-thread cycles into errors and leaves cross-thread cycles as deadlocks. Detecting those needs shared state: a global wait-for graph, a global order on actors, or bounded waiting with a fallback to asynchronous delivery.

5.4 Order

Programmers rely on the order in which messages arrive. This subsection states what order is guaranteed when `send` and `fast_send` are mixed: messages sent from one actor to another still arrive in the order sent; a `fast_send` does not wait behind messages already queued for the callee; and anything the callee sends while answering a `fast_send` is queued before whatever the caller sends after the call returns.

Proposition 13 (Order). (1) Messages sent with `send` by the same actor ι_a to the same receiver ι_b start turns of ι_b in the order ι_a sent them. Nothing is claimed about the relative order of messages from different senders. (2) A `fast_send` from ι_a to ι_b is never placed in ι_b 's mailbox: its turn starts when a `fast_send` rule applies, whatever the mailbox holds, so it can start before messages already queued for ι_b , including earlier `sends` from ι_a . While ι_b is busy, the calculus leaves open whether the pending call or the head of the mailbox starts next; a runtime that always prefers a pending `fast_send` guarantees that the call starts before every message that was in ι_b 's mailbox when it was made. (3) Messages sent during the callee's turn of a `fast_send` are enqueued before any message the caller sends after the `fast_send` returns.

Proof. (1) The sends of ι_a are totally ordered, because ι_a has at most one frame (Lemma 4); `SEND` appends and `RECEIVE` removes the head. (2) The `fast_send` rules start the callee's turn without consulting its mailbox, and `RECEIVE` does not consult pending calls; when both are enabled for ι_b the choice is free. Under the preference, each time ι_b becomes idle while the call is pending, a `fast_send` rule is applied to ι_b before any `RECEIVE` of ι_b , so no queued message of ι_b starts first. (3) The caller's frame is waiting from the `fast_send` until `FAST-RETURN`, and the callee's sends happen before that. $\square$

²Kaspar-HFT does not implement the call-chain test yet: `fast_send` takes the receiver's mutex directly, so a synchronous cycle currently deadlocks.

Part (2) is the formal counterpart of “**fast_send** bypasses the mailbox”, and it is one of the practical reasons to use **fast_send**: a call is not delayed by the backlog in the callee’s mailbox. It is not a priority mechanism — a turn in progress is never interrupted (Remark 2). The FPGA runtime of Section 7 implements the preference: each actor process serves its **fast_send** request FIFOs before its message FIFOs.

5.5 **fast_send** is not send-then-wait

A synchronous call is often implemented as a send followed by a blocking wait for the reply, and the classic results show that synchronous communication can always be encoded that way. This subsection shows that, once actors share a thread, the two are not interchangeable: there is a program in which the send-and-wait version never finishes and the **fast_send** version does. That is the formal reason **fast_send** is a primitive of its own rather than a pattern built from **send**.

Let *send-then-wait* be the pattern of the first group in Section 3: the caller sends a request with **send** and its frame then blocks until a reply message for it arrives. We model the blocking by a frame that no rule reduces until the reply has been enqueued in the caller’s mailbox; the frame stays on its thread.

Proposition 14 (Shared resources). *Let actors ι_a and ι_b share a home thread, and let ι_b ’s method return without further communication. A turn of ι_a that obtains ι_b ’s result by *send-then-wait* never completes. The same turn using **fast_send** with FAST-LOCAL completes.*

Proof. With send-then-wait, the request is enqueued in ι_b ’s mailbox and ι_a ’s frame stays on the shared thread θ . RECEIVE for ι_b requires θ to have an empty stack, which it never has again, so the reply is never produced. With **fast_send**, FAST-LOCAL pushes ι_b ’s turn on θ above ι_a ’s frame; it terminates and FAST-RETURN resumes ι_a . $\square$

Figure 4(b) illustrates the proposition. The encodings of synchronous into asynchronous communication [8, 4] are sound in a setting where every process progresses independently. In a setting with a bounded number of execution resources — actor groups, ABS cogs, E vats, a pinned thread per core, a fixed number of FPGA processes — **fast_send** and send-then-wait differ in whether they complete. The proposition also shows why a synchronous call inside one scheduling unit, as in ABS cogs and E vats, has to run on the caller’s processor: inside the unit, send-then-wait cannot complete.

6 Simulation and debugging from a global queue

Deterministic replay runs recorded external inputs through the same actor handlers used in production; in a trading system this is a backtest. A debugger needs the same thing plus the ability to stop, step and inspect. Both are easiest when every actor in the system — whatever language it is written in and whatever substrate it normally runs on — is driven from a single queue by a single thread: every message goes into the queue, receivers take their messages from it, and the execution is one deterministic sequence that can be replayed, paused and inspected. The Kaspar-HFT actor group, which co-schedules actors behind one mailbox, already serves as such a simulator for CPU actors [10].³ The question is whether **fast_send** breaks the correspondence between that simulator and the concurrent system.

³The cross-language, cross-substrate global queue described in this section is not implemented in Kaspar-HFT yet; CPU actors in one group already run this way.

The answer has two parts, and the section is organised around them. A simulator is useful for finding bugs if everything it shows could also happen in the real system; otherwise a failure seen in simulation might be an artefact of the simulator. It is useful for ruling bugs out only if, in addition, everything the real system can do also shows up in simulation. Section 6.1 defines the simulator and the real system precisely, as two variants of the calculus. Section 6.2 proves the first property, with `fast_send` included. Section 6.3 shows that the second fails, says which behaviour the simulator misses, and gives an offline check that covers the most important of it, deadlock.

6.1 The global-queue semantics

To compare the simulator with the real system we describe both in the same terms. Both are the calculus of Section 4, with different resources. The simulator has a single thread and a single queue shared by every actor. The real system, which we call the production semantics, has any number of threads, a mailbox per actor, and either form of `fast_send` at each call site. One condition on the real system is needed for the comparison to work; we state it here, and list which of the runtimes in this paper meet it.

The global-queue semantics GQ is the calculus of Section 4 with one thread θ_0 , the home thread of every actor, and with per-actor mailboxes replaced by one FIFO queue Q of pairs (ι_b, m) :

- `SEND` appends $(\iota_b, \mathcal{M}\langle msg, \text{pass}(\bar{v}) \rangle)$ to Q ;
- `RECEIVE-GQ`: when θ_0 has an empty stack, remove the head (ι_b, m) of Q and start ι_b 's turn for m ;
- `fast_send` uses `FAST-LOCAL` on θ_0 .

An actor that normally runs on another substrate takes part through its method bodies alone: an FPGA actor's turn is its step function, executed by the simulator in software; a Rust actor's turn is its handler, called across the language boundary. By receiver transparency, the body is the same whatever starts the turn.

The production semantics P is the calculus of Section 4 with any assignment of actors to threads and any choice of `FAST-LOCAL` or `FAST-REMOTE` per call site, subject to one condition on the resources that `FAST-REMOTE` uses:

Assumption 15 (Dedicated remote resources). A thread that serves as the home thread of a `FAST-REMOTE` callee is the home of that actor only.

Which runtimes satisfy it:

- *CPU to FPGA, and between FPGA processes*: an FPGA actor that has a process to itself satisfies the assumption. Between FPGA processes the request goes over a FIFO straight into the callee's `fast_send` port and the reply over a FIFO straight back. An actor held inside another actor's process and reachable from outside it shares that process, so the assumption fails for calls to it from other processes.
- *Within the CPU runtime*: `fast_send` is `FAST-LOCAL`, so the assumption does not apply.
- *FPGA to CPU*: the CPU handler runs on a host thread dedicated to the calling FPGA actor, which waits for it; that thread acts as part of the caller's resource, so the call is `FAST-LOCAL` and the assumption does not apply. A separate host thread only reads from the card and never runs a handler, so replies are always delivered.

- *Another machine*: the assumption holds if the remote actor has a thread of its own, and fails if that thread also serves other actors, for example a group.

Where the assumption fails, Theorem 16 does not cover the calls concerned.

6.2 Soundness

This subsection proves that the simulator never shows anything the real system cannot do. The idea of the proof is to replay a simulator run in the real system step by step, and to check that each step the simulator takes is also allowed in the real system. The one step where this is not obvious is a **fast_send**: in the simulator the callee is always free when called, unless it is already on the current call chain, and we must show that the real system can find it free too.

Say that two executions *agree* if every actor starts the same turns in the same order, with α -equivalent messages, and every turn ends with an α -equivalent value and heap.

Theorem 16 (Trace soundness of the global queue). *Under Assumption 15, for every finite execution of GQ from a given initial configuration there is an execution of P from the corresponding initial configuration that agrees with it: the traces of GQ are included in those of P . In particular, a configuration in which a turn is stuck, or an error state, reached in GQ has an agreeing counterpart in P .*

Proof. We build the P execution step by step, maintaining the invariant that at every point where θ_0 has an empty stack in GQ , every thread of P has an empty stack, the heaps agree, and each actor's mailbox in P is the projection of Q onto that actor, in order.

Receive. GQ takes the head (ι_b, m) of Q . By the invariant, m is the head of ι_b 's mailbox in P , since every earlier pair for ι_b in Q has already been removed. ι_b 's home thread is idle and ι_b is not busy, so **RECEIVE** applies in P to the same message.

Within a turn. P reduces only the thread that holds the corresponding frame, mirroring each GQ step: actor-local steps are identical; **SEND** appends to the receiver's mailbox, which preserves the projection invariant.

fast_send. In GQ , **FAST-LOCAL** to ι_b applies if and only if ι_b has no frame on θ_0 's stack. In the mirrored P execution the only frames in existence are the ones mirroring that stack, so ι_b is busy in P exactly when it is busy in GQ . If P uses **FAST-LOCAL** for the call, it applies. If it uses **FAST-REMOTE**, the rule also needs ι_b 's home thread to be idle. By Assumption 15 a frame reaches that thread only as a turn of ι_b or stacked above one; ι_b has no frame, so the thread is idle. The callee's turn then proceeds as above, and **FAST-RETURN** applies in both.

If GQ is stuck on a **fast_send** to a busy actor, the mirrored P frame is blocked on the same actor. Agreement of values and heaps follows from Theorem 7, since the mirrored turns receive the same replies. $\square$

Corollary 17 (What the simulator shows is real). *Every value a **fast_send** returns in GQ , every assertion failure, error state or same-thread **fast_send** cycle it reaches, can occur in the concurrent system. A GQ run is deterministic given its inputs, so it can be replayed exactly.*

So **fast_send** does not break the global-queue simulation. What makes the argument go through is that in a single-threaded run the callee of a **fast_send** is idle unless it is already on the current call chain: the simulator only ever takes the branch of **fast_send** in which the callee was free, which is one of the branches the concurrent system can take. It is not obvious in general: without Assumption 15, a **FAST-REMOTE** callee whose resource is occupied by another actor blocks in P where GQ proceeds, and the simulator would then show behaviour the concurrent system cannot produce.

Table 5. The global queue as a simulator of the concurrent system (under Assumption 15).

Question	Answer	Result
Can it produce an execution the concurrent system cannot?	no	Theorem 16
Can it reproduce every execution of the concurrent system?	no	Proposition 18
Can it exhibit a deadlock needing two mailbox turns at once?	no	Proposition 18
Can it exhibit a same-thread fast_send cycle?	yes	Theorem 16
Is a run reproducible from its inputs?	yes	single queue, single thread

6.3 What the simulator cannot show

A simulator that runs one handler at a time cannot reproduce behaviour that needs two handlers to be in progress at once. This subsection gives two concrete examples of such behaviour, one of them a deadlock, and so shows that the simulator is sound but not complete: a bug it shows is real, but a run without bugs does not prove there are none. For deadlocks, which matter most, we then give a check that does not depend on running the right inputs: look at which actors can **fast_send** which, and make sure there is no circle.

Proposition 18 (Incompleteness). *There are executions of P that agree with no execution of GQ . Two examples: (1) the deadlock of Figure 4(a), where the turns of X and Y , both started from mailboxes, never end; (2) a turn of ι_a that sends m to ι_b and then makes a **fast_send** to ι_b , where ι_b 's home thread runs the turn for m before the **fast_send** applies, so the call sees ι_b 's state after m .*

Proof. In GQ , RECEIVE- GQ needs an empty stack, so at most one turn started from a mailbox is unfinished at any time, and every other turn in progress is a **fast_send** callee on the same stack. (1) has two unfinished turns started from mailboxes, so an agreeing GQ execution would need both, which it cannot have. In (2), m enters Q during ι_a 's turn and can start only after that turn ends, so in every GQ execution ι_b starts the turn for the **fast_send** before the turn for m ; in the P execution it starts them in the other order. $\square$

Table 5 summarises what the global queue does and does not give. Replay can therefore confirm bugs but cannot rule out the two classes above. The first can be approached offline:

Proposition 19 (Cycles in the call graph). *Let G be the directed graph on actors with an edge $\iota_a \rightarrow \iota_b$ whenever some turn of ι_a can evaluate a **fast_send** to ι_b . If G is acyclic, no execution of P reaches a wait-for cycle.*

Proof. The frames on any thread, bottom to top, belong to a path in G , since each frame above the bottom one was pushed by a **fast_send** from the frame below (FAST-LOCAL); a FAST-REMOTE callee's frame extends the same path onto another thread. A wait-for edge $\theta \rightarrow \theta'$ adds the edge from the actor on top of θ to an actor already on the path held by θ' . Following a wait-for cycle and concatenating these paths yields a cycle in G . $\square$

A GQ run can record the **fast_send** edges it executes. The recorded graph is a subgraph of G — it contains only the calls the inputs exercised — so a cycle in it is a real risk to be removed, and the absence of a cycle is evidence only for the paths the backtest covered. A static analysis of the call sites would give all of G .

Time. *GQ* reproduces what happens, not when. Latency on each substrate is a separate question (Section 10).

7 Mapping `send` and `fast_send` onto substrates

This section asks what the model looks like when actors run on real hardware: CPU threads, a Rust runtime, an FPGA and, as a proposal, a GPU. The calculus describes what `send` and `fast_send` mean; each substrate must provide some mechanism that behaves that way. The point we make is that, with `fast_send` in the model, every substrate has a natural mechanism for each primitive, and on an FPGA `fast_send` is what makes the mapping complete.

An FPGA design is built from two kinds of connection. Independent hardware processes run side by side and exchange data through FIFOs. Within one process, computations are fused into a single pipeline: one stage’s output feeds the next directly, with no FIFO and no handshake, which is how FPGA designs reach their latency and throughput. Actors map onto processes and `send` onto FIFOs, as one would expect. Fused logic, however, has no counterpart in a model with only asynchronous `send`, and it cannot be given one by building request/reply out of `send`: two actors fused into one process share that process, and Proposition 14 shows that a `send` followed by a wait between actors that share their execution resource never completes. With `fast_send`, a call between fused actors is simply the callee’s handler inlined into the caller’s pipeline, and a call to another process is a request over a FIFO with the caller’s process stalled until the reply arrives. Both connections the hardware offers then correspond to a primitive of the model.

The calculus leaves open what a thread is and how a mailbox is built. Each substrate supplies both, and each supplies a distinct mechanism for each primitive (Table 6).

On an FPGA the two `fast_send` rules correspond to two different hardware arrangements. When the callee is compiled into the caller’s hardware process, its handler becomes part of the caller’s pipeline and runs on the caller’s resource: FAST-LOCAL. When the callee is a separate process, it has its own resource, which the caller waits on: FAST-REMOTE. In the runtime described in Section 8 each actor is its own process, and FAST-LOCAL applies to actor objects that a process contains.

The pattern is the same everywhere. `send` is a queue between two independently scheduled resources. `fast_send` is either an inline call on the caller’s resource, where the substrate lets one resource run another actor’s handler, or a blocking request to a resource dedicated to the callee, where it does not. The calculus of Section 4 covers both with one semantics, and Theorems 6, 7 and 10 hold for any mixture.

A `fast_send` is available between any two substrates where the caller can wait. A CPU thread can wait on a condition. An FPGA process can wait on a blocking stream read, which stalls that process alone while the rest of the chip keeps running. What differs between substrate pairs is the cost of the wait.

8 Practical implementation in Kaspar-HFT

Kaspar-HFT has actors on three runtimes — C++ on CPU threads, Rust, and an FPGA runtime that currently runs as a software model of the hardware — and a GPU runtime is planned. Each runtime has its own actors, written in its own language; an actor is not recompiled from one runtime to another. What is shared is the model: on every runtime an actor holds references to other actors, whether they run on the same runtime or another, and uses the same two primitives, `send` and `fast_send`, and the same `reply`. The extended actor model of Sections 4 and 5 is the semantics of

Table 6. `send` and `fast_send` on each substrate and between substrates, and the rule of Section 4 each `fast_send` implements.

Where	<code>send</code>	<code>fast_send</code>
CPU, one process	message into the receiver’s mailbox; a thread per actor or per group	callee’s handler runs on the caller’s thread under the callee’s lock (FAST-LOCAL)
C++ ↔ Rust, one process	message converted and enqueued through a C ABI bridge	callee’s handler runs on the caller’s thread through the bridge (FAST-LOCAL)
FPGA, one process	a queue inside the process	callee is an actor object inside the caller’s process; its handler is inlined into the caller’s pipeline (FAST-LOCAL)
FPGA, between processes	a FIFO from sender to receiver	a FIFO to the callee’s <code>fast_send</code> port, served before its mailbox; the reply over a FIFO back to the caller; the caller’s process waits (FAST-REMOTE)
CPU → FPGA	envelope over PCIe into the receiver’s FIFO from the host	envelope to the actor’s <code>fast_send</code> port, served before its mailbox; the CPU thread waits for the reply (FAST-REMOTE)
FPGA → CPU	envelope over PCIe; a host thread enqueues it	request over PCIe; a host thread dedicated to the calling FPGA actor runs the CPU handler; the FPGA process stalls until the reply returns (FAST-LOCAL, on a thread acting for the caller)
GPU (proposed)	ring buffer in device memory	device function call inside one kernel (FAST-LOCAL); host calls wait on a reply ring (FAST-REMOTE)
Another machine	network message	request/reply RPC; the caller waits (FAST-REMOTE)

all of them, and of any mixture of them, with each `fast_send` implemented by FAST-LOCAL or FAST-REMOTE as Table 6 shows. Table 7 separates what is formalised, what is implemented and how each part has been exercised. The rest of this section describes each runtime and then the facility for debugging and simulation.

CPU runtime. The Kaspar-HFT C++ runtime [10] implements actors with a mutex-protected turn, a selectable mailbox per actor, `send`, `fast_send` and groups. `fast_send` takes the callee’s lock on the caller’s thread and runs its handler inline; a reply is returned as a value. The delivery flag is private to the framework, but some message fields — the queue depth, the “mailbox empty” flag and the destination — are filled differently for the two deliveries, so a handler that reads them can infer how it was reached. Receiver transparency in Kaspar-HFT is therefore a property of how handlers are written, as in Remark 3, not one the framework enforces. A mailbox may also be configured to busy-poll instead of sleeping, which changes the cost of `send` between threads but not its semantics.

Table 7. Status of each part in Kaspar-HFT. “Semantics” refers to this paper.

Part	Semantics	Implementation	Exercised by
C++ CPU runtime: <code>send</code> , <code>fast_send</code> , groups	§4–5	implemented	tests; the Kaspar-HFT trading system [10]
Rust runtime; C++ $\leftrightarrow$ Rust <code>send</code> and <code>fast_send</code>	§4–5	implemented; cross-language <code>fast_send</code> returns no reply yet	tests
FPGA runtime and CPU bridge	§4–5	implemented as a software model	unit tests in software
FPGA runtime on hardware	§4–5	not yet synthesised	—
GPU runtime	Table 6	design	—
Global queue and sequencer	§6	design; single-runtime instance: actor groups	—
Per-thread call-chain check	Proposition 12	design	—

Rust runtime and interop. A separate Rust actor runtime hosts Rust actors. C++ and Rust actors reach each other through generated C-ABI bridges, from one message schema; both `send` and `fast_send` work across the boundary.⁴

FPGA runtime. The FPGA runtime is written in Vitis HLS C++. Each actor is a hardware process; its handlers are selected by a switch on the message identifier. Two actors that communicate are joined by FIFOs from one to the other — for messages, for `fast_send` requests and for their replies — and a sender writes straight into the FIFO that leads to the receiver. A table fixed when the design is built tells a sender which FIFO leads to a given actor. Each actor serves its `fast_send` request FIFOs before its message FIFOs, and a remote `fast_send` waits on the reply FIFO from its callee. A host link connects the design to PCIe. On the CPU, an `ActorRef` can refer to an FPGA actor; `send` and `fast_send` on it write envelopes to the card. On the way back, a host thread reads the card and delivers messages and replies, and `fast_send` requests from each FPGA actor are run by a host thread dedicated to that actor; both pass a stand-in for the FPGA actor as the sender so that ordinary replies return to the chip. The design runs today in software, with one thread per hardware process, and its unit tests check `send` and `fast_send` between FPGA processes, `fast_send` inside a process, `send` and `fast_send` in both directions between CPU and FPGA actors, ordering, error reporting, and the priority of `fast_send` request FIFOs over message FIFOs.⁵

GPU runtime. The same structure maps onto a GPU: actors as warps or thread blocks of a persistent kernel, mailboxes as device-memory rings, `fast_send` inside the kernel as a device function call, and a host bridge over rings in pinned memory. This is a proposed mapping; it has not been built.

⁴Kaspar-HFT does not return `fast_send` replies across the language boundary yet; a cross-language `fast_send` delivers the message and returns no value.

⁵The FPGA runtime has not yet been synthesized or run on a card.

Global queue and sequencer. Debugging and simulation across runtimes will be provided by a global queue and a *sequencer*, which implement the global-queue semantics GQ of Section 6.⁶ In this mode every runtime hands each message it sends, whatever its destination, to the sequencer instead of delivering it. The sequencer gives the message a global sequence number and appends it to the global queue. It then delivers messages one at a time: it takes the head of the queue, passes it to the runtime of the receiving actor, and waits until that runtime reports that the turn has ended, including every `fast_send` the turn made, before it delivers the next. Each runtime therefore needs only an adapter with two operations — accept one delivery, report the end of the turn — and keeps its own actors, handlers and `fast_send` mechanism unchanged; an FPGA actor can be driven on the card or in software simulation of its hardware process.

Because deliveries are serialized, a debugger can step through the queue one message at a time and see, before each step, which actor on which runtime will run next and with what message; between steps it can inspect the state of any actor on any runtime, and it can stop on a given actor, message type or runtime. Theorem 16 is what makes this useful, for every call that satisfies Assumption 15: the sequence the debugger shows is one the concurrent system can produce, so a fault found by stepping is real. The same sequence, recorded, replays a backtest exactly, and the `fast_send` calls it executes give the call graph of Proposition 19. What the sequencer cannot show is behaviour such as that of Proposition 18, which needs more than one turn started from a mailbox in progress at once.

9 Related work

Actor semantics. The operational semantics of Agha, Mason, Smith and Talcott [2] and the four-family semantics of De Koster and De Meuter [6] contain only asynchronous communication. We extend the latter. Our isolation and liveness results are stated against their isolated turn principle.

Synchronous calls. ABCL/1 [14] and ABS [9] provide synchronous calls between concurrent objects that block the caller while the callee runs on its own processor, and ABS adds reentrant synchronous calls inside a cog. E [11] restricts immediate calls to one vat. Extended Rebeca [12, 13] is the closest actor language with a formal semantics for synchronous messages, and differs from `fast_send` on each point of Table 3. Its synchronous messages are rendezvous: the receiver must execute an explicit `receive` statement, no handler is invoked, and data flows only from sender to receiver, so there is no reply value. They are confined to one component, while `fast_send` addresses any actor. A rebec blocked in a rendezvous is blocked inside a method, whereas a `fast_send` callee’s turn is an ordinary handler that runs to completion. Its semantics has no execution resources, so the questions answered here for `fast_send` — behaviour when the callee’s resource is shared (Proposition 14) or occupied (Assumption 15), and single-queue simulation (Section 6) — do not arise in it; its aim is compositional model checking. We therefore do not claim that synchronous messages in an actor language are new; the contribution is the particular resource-sensitive primitive and its properties.

One can ask whether `fast_send` could be encoded with rendezvous. An encoding would have the callee wait at a `receive` for the request, run the handler’s code, and hand the result back through a second rendezvous. The callee’s code then runs on the callee’s side, not on the caller’s resource, and the caller’s blocked computation keeps its resource while it waits; when the two share one resource,

⁶Kaspar-HFT does not implement the cross-runtime global queue and sequencer yet.

the argument of Proposition 14 applies unchanged and the encoding deadlocks where FAST-LOCAL completes. The survey of active-object languages by De Boer et al. [5] maps this design space.

Encodings. Honda and Tokoro [8] and Boudol [4] encode synchronous in asynchronous communication; Proposition 14 locates where such encodings stop applying once resources are bounded.

Actors on hardware. CAL and the StreamBlocks compiler [3] compile dataflow actors to both CPUs and FPGAs. Dataflow actors fire on token availability and have no synchronous call; our FPGA runtime targets the message-passing actor model with both primitives.

10 Limitations and future work

- The calculus and proofs are on paper. A mechanisation, for instance by extending the PLT Redex models of [6], would check them.
- The calculus has no failures; a **fast_send** to an actor on a failed resource waits forever. Timeouts and their interaction with isolation are open.
- The calculus’s mailboxes are unbounded, so **send** never blocks. In hardware the FIFOs between FPGA actors, and between the host link and each actor, have a fixed depth, and a write into a full FIFO waits. Two actors that send to each other can then both wait for ever, and one full FIFO from the host link can hold up all traffic from the CPU behind it. Theorem 10 does not cover this blocking. Modelling bounded mailboxes, and sizing FIFOs so that a design cannot reach it, are open.
- Detecting cross-thread cycles at run time needs shared state (Proposition 12); its cost on the hot path is not measured.
- Performance is outside the scope of this paper. The cost of **send** and **fast_send** on the C++ runtime is measured in [10]; measuring it across the language boundary, on the FPGA runtime once it is synthesised and run on a card, and between runtimes, is future work, as is the GPU runtime itself.
- The global queue and sequencer of Section 8 are designs; building them, and a static analysis that computes the call graph of Proposition 19, would turn Theorem 16 and Proposition 19 into tools.

11 Conclusion

We proposed adding a synchronous send, **fast_send**, to the actor model as a second primitive alongside asynchronous send: the caller waits while the receiver handles the message, and gets the reply back as the return value. We gave it a formal semantics with explicit execution resources and proved what adding it keeps and what it costs. Isolation is kept. Determinism holds once the replies a handler receives count as its input. Liveness becomes conditional: with terminating handlers and fair scheduling, a handler can be stuck only when actors wait on each other in a circle, and without **fast_send** the original guarantee returns. When actors share a thread, **fast_send** completes where a send followed by a wait deadlocks, so it is a primitive of its own rather than a pattern built from **send**. A single global queue remains a sound simulator with **fast_send**: replay or a debugger that

runs every actor, in any language and on any substrate, from one queue shows only behaviour the concurrent system can produce, though not all of it.

With `fast_send` in the model, actors map naturally onto hardware. On an FPGA, `send` is a FIFO between hardware processes and `fast_send` is either logic fused into the caller’s process or a blocking request to another process; actors fused into one process can call each other only with `fast_send`. The same two mechanisms — a queue, and an inline or blocking call — implement the two primitives on CPU threads and in the Rust runtime, and a GPU runtime would follow the same pattern.

References

- [1] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.
- [2] G. A. Agha, I. A. Mason, S. F. Smith, and C. L. Talcott. A foundation for actor computation. *Journal of Functional Programming*, 7(1):1–72, 1997.
- [3] E. Bezati, M. Emami, J. Janneck, and J. Larus. StreamBlocks: A compiler for heterogeneous dataflow computing (technical report). arXiv:2107.09333, 2021.
- [4] G. Boudol. Asynchrony and the π -calculus. Technical Report 1702, INRIA Sophia-Antipolis, 1992.
- [5] F. de Boer, V. Serbanescu, R. Hähnle, L. Henrio, J. Rochas, C. C. Din, E. B. Johnsen, M. Sirjani, E. Khamespanah, K. Fernandez-Reyes, and A. M. Yang. A survey of active object languages. *ACM Computing Surveys*, 50(5), 2017.
- [6] J. De Koster and W. De Meuter. A formal specification for half a century of actor systems. Manuscript, Vrije Universiteit Brussel, 2023.
- [7] C. Hewitt, P. Bishop, and R. Steiger. A universal modular ACTOR formalism for artificial intelligence. In *IJCAI*, 1973.
- [8] K. Honda and M. Tokoro. An object calculus for asynchronous communication. In *ECOOOP ’91*, LNCS 512, pages 133–147, 1991.
- [9] E. B. Johnsen, R. Hähnle, J. Schäfer, R. Schlatter, and M. Steffen. ABS: A core language for abstract behavioral specification. In *Formal Methods for Components and Objects (FMCO 2010)*, LNCS 6957, pages 142–164. Springer, 2011.
- [10] V. Maciejewski. Adapting the actor model of concurrency for high-frequency trading: synchronous message delivery (`fast_send`) and a tick-to-book latency study. arXiv:2609.21173, 2026.
- [11] M. S. Miller, E. D. Tribble, and J. Shapiro. Concurrency among strangers: programming in E as plan coordination. In *Trustworthy Global Computing*, 2005.
- [12] M. Sirjani, A. Movaghar, A. Shali, and F. S. de Boer. Extended Rebeca: a component-based actor language with synchronous message passing. In *Application of Concurrency to System Design (ACSD’05)*, pages 212–221, 2005.

- [13] M. Sirjani, F. S. de Boer, and A. Movaghar. Modular verification of a component-based actor language. *Journal of Universal Computer Science*, 11(10):1695–1717, 2005.
- [14] A. Yonezawa, J.-P. Briot, and E. Shibayama. Object-oriented concurrent programming in ABCL/1. In *OOPSLA '86*, pages 258–268, 1986.